

Chapter 5

Arbres

(Version du March 3, 2026)

5.1 Arbres binaires hétérogènes

5.1.1 Définitions

On peut définir rigoureusement la notion d'arbre par induction structurelle:

Définition (Arbre binaire)

Soient $\mathcal{N}$ et $\mathcal{F}$ deux ensembles disjoints et ne contenant, ni l'élément $($, ni l'élément $)$, ni l'élément $,$. Soit E l'ensemble des mots sur l'alphabet $\mathcal{A}$ constitué de la réunion des ensembles $\mathcal{N}$, $\mathcal{F}$ et éléments $($ et $)$. Pour tout $n \in \mathcal{N}$, on définit la fonction de E^2 vers E , $f_n : (G, D) \mapsto (G, n, D)$. On appelle ensemble des arbres binaires, le sous-ensemble de E défini par induction structurelle à partir de l'ensemble de base $\mathcal{F}$ et des fonctions f_n pour $n \in \mathcal{N}$. On note cet ensemble $\mathcal{T}(\mathcal{F}, \mathcal{N})$ ou plus simplement $\mathcal{T}$.

On appelle arbre binaire un élément de cet ensemble.

✓ L'ensemble des arbres binaires est donc le plus petit sous-ensemble de E contenant les éléments de $\mathcal{F}$ et stable par les applications $(G, D) \mapsto (G, n, D)$.

Proposition (Vérif. d'un prédicat par induction sur les Arbres)

Soit $\mathcal{P}$ un prédicat défini sur l'ensemble E .

(-) Si $\mathcal{P}(f)$ est vrai pour tout élément de $f \in \mathcal{F}$;

(-) et si pour tout arbres G et D , et tout élément n de $\mathcal{N}$, on a l'implication $\mathcal{P}(G) \wedge \mathcal{P}(D) \implies \mathcal{P}((G, n, D))$,

alors pour tout arbre T , on $\mathcal{P}(T)$ vraie.

Pour prouver ce théorème, nous avons envie de se ramener au théorème du chapitre 2, on a envie pour cela de définir les fonctions qui à un arbre qui n'est pas une feuille associent les sous-arbre de gauche et de droite. Cela n'est pas si simple *a priori*.

Soit $w = w_0 \dots w_{p-1}$ un mot de longueur $p \in \mathbb{N}^*$ de E .

Pour tout $i \in \llbracket 0, p-1 \rrbracket$, on note $\delta_i(w)$ la différence entre le nombre de parenthèses ouvrantes et le nombre de parenthèses fermantes parmi les caractères $w_0, \dots, w_i$. On note $\delta(w) = \delta_{p-1}(w)$. Donc $\delta(w)$ est simplement la différence entre le nombre de parenthèses ouvrantes et fermantes dans le mot w .

On prouve alors facilement par induction sur les arbres, que pour tout arbre T on a $\delta(T) = 0$, on prouve alors ensuite que si un arbre s'écrit sous la forme (G, n, D) avec G et D deux arbres, et si i le numéro du caractère n dans le mot $w = (G, n, D)$ (de longueur p), alors i est l'unique élément de $\llbracket 0, p-1 \rrbracket$ tel que $w_i \in \mathcal{N}$ et $\delta_i(w) = 1$, on en déduit que pour tout arbre T qui n'est pas une feuille, il existe une unique écriture de T sous la forme (G, n, D) où G et D sont des éléments de $\mathcal{T}$.

On peut ainsi définir trois fonctions ϕ_1 , ϕ_2 et r sur $\mathcal{T} \setminus \mathcal{F}$, en posant pour tout arbre T de la forme (G, n, D) :

$$r(T) = n, \quad \phi_1(T) = G \quad \text{et} \quad \phi_2(T) = D.$$

On rappelle que l'ensemble E des mots est un ensemble bien fondé pour la relation, "être un sous-mot" de, notée $\preceq$. (On dit qu'un mot w est un sousmot de w' si le mot w est une tranche obtenue en sélectionnant un certain nombre de lettres consécutives de w'). La restriction de cette relation à $\mathcal{T}$ définit encore un ensemble bien fondé, on la note encore $\preceq$. On remarque alors que pour tout élément de $\mathcal{T} \setminus \mathcal{F}$, on a $\phi_1(T) \prec T$, $\phi_2(T) \prec T$ et $r(T) \prec T$, on peut donc utiliser les propriétés sur les définitions de fonction par induction dans un ensemble bien fondé, pour en déduire la propriété suivante:

Proposition 5.1 (Définition d'une fonc. par induction sur les arbres)

Soit B un ensemble. Soient $g : \mathcal{F} \mapsto B$ une application et $h : \mathcal{N} \times B \times B \mapsto B$ une autre application.

Alors il existe une unique fonction $\psi : \mathcal{T} \mapsto B$ telle que:

- (-) $\forall f \in \mathcal{F}, \psi(f) = g(f)$,
- (-) et $\forall T \in \mathcal{T} \setminus \mathcal{F}, \psi(T) = h(r(T), \psi(\phi_1(T)), \psi(\phi_2(T)))$

Autrement dit, on peut se contenter, pour définir une fonction ψ sur les arbres, de définir cette fonction sur les feuilles et de définir comment obtenir (G, n, D) à partir de $\psi(G)$ et $\psi(D)$.

Plus pratiquement, on se contente des définitions suivantes:

Définition 5.2 (Inductive d'arbre binaire)

Soient $\mathcal{N}$ et $\mathcal{F}$ deux ensembles. On définit inductivement un arbre binaire dont les noeuds sont étiquetés par $\mathcal{N}$ et les feuilles étiquetées par $\mathcal{F}$ par:

T est un arbre si:

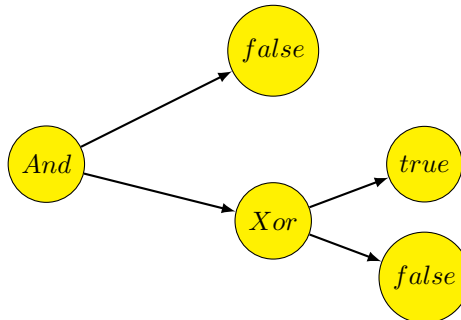
- Il existe f dans $\mathcal{F}$ telle que $T = f$,
- ou s'il existe n dans $\mathcal{N}$ et G et D deux arbres tels que $T = (G, n, D)$.

On note $\mathcal{A}(\mathcal{F}, \mathcal{N})$ l'ensemble des arbres ainsi définis.

Exemples:

1. Arbre associé à une expression arithmétique avec seulement des opérateurs binaires. On peut utiliser l'ensemble $\mathcal{N} = \{+, \times\}$ et l'ensemble $\mathcal{F} = \mathbb{R}$, les éléments de $\mathcal{A}(\mathbb{R}, \mathcal{N})$ sont alors naturellement associés à une expression arithmétique.
2. Arbre associé à une expression logique avec seulement des opérateurs binaires. On peut utiliser l'ensemble $\mathcal{N} = \{xor, and, or\}$ et l'ensemble $\mathcal{F} = \{true, false\}$.

Exemple:



Vocabulaire: Les arbres de la forme f pour $f \in \mathcal{F}$ sont appelés des feuilles, on les appelle aussi noeuds externes. Un arbre qui est une feuille n'a pas de noeud interne.

Tout arbre T non feuille admet une unique écriture de la forme:
 $T = (Tg, r, Td)$ avec $r \in \mathcal{N}$ et Tg et Td deux arbres.

- On dit que r est la racine de T ,
- Tg est le fils gauche et Td est le fils droit.
- T est le père de Tg et de Td .
- On dit que T ou r est un noeud interne de T , les autres noeuds internes de T sont ceux de Tg et ceux de Td .

On choisit parfois de numéroter les noeuds de l'arbre, les noeuds et les feuilles de l'arbre ont alors des étiquettes de la forme $R = (r, n)$ avec $r \in \mathcal{N}$ ou $r \in \mathcal{F}$ et n un numéro entier permettant de distinguer les noeuds de l'arbre. Les valeurs de n dans l'arbre sont distinctes deux à deux. On supposera dans la suite que les étiquettes sont distinctes deux à deux (quitte à utiliser l'astuce précédente). On parle alors de noeuds de l'arbre. On confond souvent le noeud d'un arbre et le sous arbre qui lui est rattaché.

Si T est une feuille, alors T n'a pas de noeud interne, si T est de la forme $T = (Tg, R, Td)$ avec Tg et Td deux arbres, alors l'ensemble des noeuds internes de T est formé par réunion de R et des ensembles des noeuds internes de Tg et Td .

Définition 5.3 (Constructive)

Soient $\mathcal{F}$ et $\mathcal{N}$ deux ensembles. On définit la suite d'ensembles suivants:

$E_0 = \mathcal{F}$.

$\forall n \in \mathbb{N}, E_{n+1} = \left(\left(\bigcup_{k=0}^n E_k \right) \times \mathcal{N} \times E_n \right) \cup \left(E_n \times \mathcal{N} \times \left(\bigcup_{k=0}^n E_k \right) \right)$. Alors $\bigcup_{n \in \mathbb{N}} E_n$ est l'ensemble $\mathcal{A}(\mathcal{F}, \mathcal{N})$ des arbres dont les feuilles sont étiquettées par $\mathcal{F}$ et les noeuds par $\mathcal{N}$.

Structure de Donnée abstraite:

Les types `feuille` et `noeud` étant définis, on utilise le type de données arbre binaire `tree` avec la signature:

Constructeurs:

`creer_feuille f` crée un arbre réduit à une feuille, c'est-à-dire l'arbre f , c'est une fonction de type `feuille -> tree`.
`cons_arbre (tg,x,td)` crée un arbre dont la racine est x et les sous-arbres de gauche et de droite tg et td , c'est une fonction de type `tree * noeud * tree -> tree`.

Sélecteurs:

`racine t` renvoie l'étiquette de la racine de T (lorsque T n'est pas une feuille), c'est une fonction de type `tree -> noeud`.
`feuille t` renvoie l'étiquette de la feuille de T quand T est une feuille, c'est une fonction de type `tree -> feuille`.
`fils_gauche t` renvoie le fils gauche de T , c'est une fonction de type `tree -> tree`.
`fils_droit t` renvoie le fils droit de T , c'est une fonction de type `tree -> tree`.

Prédicat:

`est_feuille t` renvoie la valeur `true` si T est une feuille et `false` sinon.

Le lecteur complètera de lui-même ce poly en donnant les axiomes correspondants.

5.1.2 Implémentation CAML

On suppose les type `feuille` et `noeud` déjà définis. On définit alors le type `tree` par:

```
type tree = F of feuille | N of tree* noeud *tree;;
```

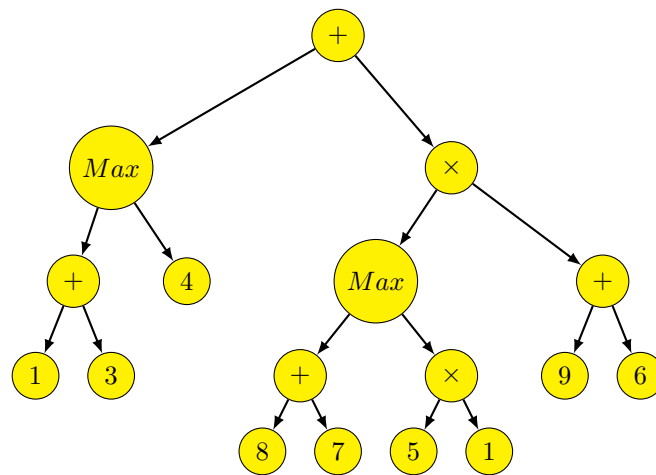
Par exemple, pour gérer des expressions arithmétiques, on peut écrire:

```
type feuille = int;;
type noeud = Plus | Foix | Max;;

let t1 = N (F 3 , Plus, F 2);;
let t2 = N (F 9, Max, t1);;
let t3 = N (t1, Foix, t2);;
```

Le lecteur dessinera l'arbre représenté informatiquement par `t3`, c'est l'arbre naturellement associé à l'expression arithmétique $(3 + 2) * (\max(9, (3 + 2)))$.

On peut alors utiliser l'arbre `tex`, représenté par la figure ci-dessous:



Les fonctions élémentaires peuvent alors être définies par:

```
let creer_feuille f= F f;;
let cons_arbre (tg,x,td)= N (tg,x,td);;

let racine t=
  match t with
  N (tg,x,td)-> x
  |_-> failwith("pas de racine");;

let feuille t=
  match t with
  F f -> f
  |_-> failwith("l'arbre n'est pas une feuille");;

let fils_gauche t=
  match t with
  N (tg,x,td)-> tg
  |_-> failwith("pas d'enfant");;

let fils_droit t=
  match t with
  N (tg,x,td)-> td
  |_-> failwith("pas d'enfant");;

let est_feuille t=
  match t with
  F f -> true
  |_-> false;;
```

5.1.3 Vocabulaire et propriétés

Dans tout ce paragraphe, $\mathcal{F}$ et $\mathcal{N}$ sont deux ensembles, et l'on appelle arbre tout élément de l'ensemble $\mathcal{A}(\mathcal{F}, \mathcal{N})$ noté abusivement $\mathcal{A}$.

Proposition 5.4 (Induction sur les arbres)

Soit $\mathcal{P}(T)$ un prédicat sur l'ensemble $\mathcal{A}$, si
 $\forall f \in \mathcal{F}, \mathcal{P}(f)$ est vrai
 et si $\forall (Tg, x, Td) \in \mathcal{A} \times \mathcal{N} \times \mathcal{A}, \quad \mathcal{P}(Tg) \text{ et } \mathcal{P}(Td) \text{ vraies} \implies \mathcal{P}((Tg, x, Td))$ est vraie,
 alors $\forall T \in \mathcal{A}, \mathcal{P}(T)$ est vrai.

Proposition 5.5 (Application définie par induction sur les arbres)

Soit B un ensemble et ψ une application de $\mathcal{F}$ vers B et φ une application de $B \times \mathcal{N} \times B$ vers B , alors il existe une unique application Ψ de $\mathcal{A}$ vers B telle que:

- $\forall f \in \mathcal{F}, \Psi(f) = \psi(f)$.
- $\forall (Tg, Td) \in \mathcal{A}^2, \forall x \in \mathcal{N}, \Psi((Tg, x, Td)) = \varphi(\Psi(Tg), x, \Psi(Td))$.

Proposition 5.6 (Nombre de noeuds internes)

Le nombre $ni(T)$ de noeuds internes d'un arbre $T \in \mathcal{A}$ est défini inductivement par:

- (-) S'il existe $f \in \mathcal{F}$ tel que $T = f$, alors $ni(T) = 0$,
- (-) S'il existe $Tg \in \mathcal{A}$ et $Td \in \mathcal{A}$ et $x \in \mathcal{N}$ tel que $T = (Tg, x, Td)$, alors $ni(T) = ni(Tg) + ni(Td) + 1$.

Ptitexo 1. Écrire en CAML la fonction donnant le nombre de noeuds internes d'un arbre donné en argument.

Proposition 5.7 (Nombre de feuilles)

Le nombre $nf(T)$ de feuilles d'un arbre $T \in \mathcal{A}$ est la fonction définie inductivement par:

- (-) S'il existe $f \in \mathcal{F}$ tel que $T = f$, alors $nf(T) = 1$,
- (-) S'il existe $Tg \in \mathcal{A}$ et $Td \in \mathcal{A}$ et $x \in \mathcal{N}$ tel que $T = (Tg, x, Td)$, alors $nf(T) = nf(Tg) + nf(Td)$.

Ptitexo 2. Écrire en CAML, la fonction donnant le nombre de feuilles d'un arbre donné en argument.

Proposition 5.8 (relation feuille/noeud interne)

Soit T un arbre, alors on a:

$$nf(T) = ni(T) + 1.$$

Le nombre total de noeuds (internes et feuilles) de l'arbre est

$$n(T) = 2ni(T) + 1.$$

preuve :

Il suffit de prouver ces résultats par induction sur les arbres, en utilisant la propriété 5.4. Montrons que le prédicat:

$$\mathcal{P}(T) := \quad nf(T) = ni(T) + 1.$$

est vrai pour tout arbre.

- (-) S'il existe $f \in \mathcal{F}$ tel que $T = f$, alors $ni(T) = 0$ et $nf(T) = 1$, donc la propriété $nf(T) = ni(T) + 1$ est bien vraie.

(–) Soient Tg et Td deux arbres tels que $\mathcal{P}(Tg)$ et $\mathcal{P}(Td)$ sont vraies. Soit $x \in \mathcal{N}$.

On a donc $ni((Tg, x, Td)) = ni(Tg) + 1 + ni(Td)$ et

$nf((Tg, x, Td)) = nf(Tg) + nf(Td)$. Comme $\mathcal{P}(Tg)$ et $\mathcal{P}(Td)$ sont vraies, on a donc

$nf(Tg) + nf(Td) = ni(Tg) + 1 + ni(Td) + 1$. Ce qui donne bien:

$nf((Tg, x, Td)) = 1 + ni((Tg, x, Td))$. Donc $\mathcal{P}((Tg, x, Td))$ est vraie.

(–) Cela prouve donc par induction sur les arbres que la propriété $\mathcal{P}(T)$ est vraie pour tout arbre T . $\square$

✓ On peut aussi raisonner par récurrence forte sur le nombre de noeuds internes.

Définition 5.9 (Hauteur d'un arbre)

On définit la fonction hauteur h qui à un arbre T associe sa hauteur $h(T)$ par:

- S'il existe $f \in \mathcal{F}$ tel que $T = f$, alors $h(T) = 0$.
- S'il existe $(Tg, x, Td) \in \mathcal{A} \times \mathcal{N} \times \mathcal{A}$ tel que $T = (Tg, x, Td)$, alors $h(T) = \max(h(Tg), h(Td)) + 1$.

✓ Si l'on reprend les notations de la définition 5.3, alors $h(T)$, la hauteur de l'arbre T est l'unique entier h tel que $T \in E_h$.

Ptitexo 3. Écrire la fonction CAML `hauteur` qui renvoie la hauteur d'un arbre.

Proposition 5.10 (Relation nombre de noeuds-hauteur)

Soit T un arbre binaire de hauteur h . Alors:

(–) Le nombre de feuilles nf vérifie: $h + 1 \leq nf \leq 2^h$.

(–) Le nombre de noeuds internes ni vérifie: $h \leq ni \leq 2^h - 1$. C'est-à-dire: $\ln_2(ni + 1) \leq h \leq ni$.

✓ Chacune des inégalités précédentes peut-être une inégalité pour des arbres particuliers, arbres peignes ou arbres parfaits.

✓ La taille d'un arbre peut être définie par ni , nf ou $n = ni + nf$ selon les cas. On peut aussi utiliser h .

✓ Certains algorithmes ont une complexité qui ne dépend que de h , d'où la nécessité de minimiser h avec ni constant.

✓ On appelle hauteur d'un noeud N la hauteur du sous-arbre associé.

Définition 5.11 (Ascendants, descendants)

Soit N un noeud de l'arbre T .

Les **fils** d'un noeud N sont définis par:

(–) Si $N = a \in \mathcal{F}$, alors N n'a pas de fils.

(–) Si $N = (Tg, a, Td)$, les fils de N sont les sous-arbres Tg et Td .

Un noeud N est dit **père** du noeud N' si N' est un fils de N . On définit inductivement les **descendants** d'un noeud N d'un arbre T par:

(–) Si $N = a$, alors N n'a pas de descendants.

(–) Si $N = (Tg, a, Td)$, alors les descendants de N sont les sous-arbres Tg et Td et leur descendants.

On dit que N est un ascendant de N' si N' est un descendant de N .

Proposition 5.12 (Chemin et profondeur d'un noeud)

Soit T un arbre de racine R . Soit N un noeud de T . Il existe une unique suite de noeuds distincts 2 à 2 $(n_0, \dots, n_p)$ tel que $n_0 = R$, $n_p = N$ et $\forall i \in \llbracket 1, p \rrbracket$, n_i est un fils de n_{i-1} . On dit que $(n_0, \dots, n_p)$ est le chemin de la racine au noeud N dans l'arbre T . On le note $ch(N)$. On dit que p est sa longueur et que p est la profondeur du noeud N dans l'arbre T . On la note $pr_T(N)$.

✓ La profondeur de la racine est 0.

✓ La profondeur d'un noeud est le nombre de branches de la racine au noeud ou bien le nombre de noeuds distincts de la racine rencontrés en suivant le chemin de la racine au noeud.

Proposition 5.13 (Définition inductive de la profondeur)

Si T est un arbre de racine r et N un noeud de cet arbre.

(-) $pr_T(N) = 0$ si $N = r$.

(-) $pr_T(N) = 1 + pr_{T_g}(N)$ si $T = (T_g, r, T_d)$ et si $N \in T_g$.

(-) $pr_T(N) = 1 + pr_{T_d}(N)$ si $T = (T_g, r, T_d)$ et si $N \in T_d$.

Proposition 5.14 (Caractérisation de la hauteur)

Soit T un arbre de hauteur h et p un entier avec $0 \leq p \leq h$.

La profondeur d'un noeud est comprise entre 0 et h .

La hauteur h est le maximum des profondeurs des feuilles de l'arbre.

Le nombre de noeuds de profondeur p est compris entre 1 et 2^p .

Définition 5.15 (niveau d'un noeud, d'un arbre)

Soit T un arbre de hauteur h . Soit $k \in \llbracket 0, h \rrbracket$. On appelle niveau k de l'arbre T , l'ensemble des noeuds de profondeur k . On le note $Niv_T(k)$.

Proposition 5.16 (Définition inductive d'un niveau)

(-) Soit T un arbre de racine r .

• $Niv_T(0) = \{r\}$.

• Si T_g et T_d sont deux arbres tels que $T = (T_g, r, T_d)$, alors
 $\forall k \in \llbracket 1, r \rrbracket: Niv_T(k) = Niv_{T_g}(k-1) \cup Niv_{T_d}(k-1)$.

(-) $\forall k \in \llbracket 1, h \rrbracket, 2 \leq \text{card } Niv_T(k) \leq 2^k$.

Définition 5.17 (Arbre parfait)

Un arbre binaire T de hauteur h est dit **parfait** si et seulement si les feuilles sont toutes à la profondeur h .

Définition 5.18 (Arbre complet)

Un arbre binaire complet est un arbre binaire dans lequel chaque niveau, sauf éventuellement le dernier, est complètement rempli et tous les noeuds du dernier niveau sont aussi à gauche que possible. Il peut y avoir entre 1 et 2^h noeuds au dernier niveau h .

✓ Un arbre parfait est donc toujours complet mais un arbre complet n'est pas toujours

parfait.

Proposition 5.19 (Caractérisation inductive des arbres parfaits)

Si T est réduit à une feuille, alors T est complet.

S'il existe deux arbres Tg et Td et $a \in \mathcal{N}$ tels que $T = (Tg, a, Td)$, alors T est parfait si et seulement si Tg et Td sont parfaits et $h(Tg) = h(Td)$.

Ptitexo 4. Écrire une fonction `est_parfait t` qui renvoie un booléen de valeur `true` si et seulement si l'arbre t est complet.

Proposition 5.20 (Dénombrement et arbres parfaits)

L'arbre T de hauteur h est parfait si et seulement si tous les noeuds de hauteur inférieure à $h - 1$ sont des noeuds internes (et donc ont exactement deux fils).

Un arbre T de hauteur h est parfait si et seulement si il a $nf = 2^h$ feuilles.

Un arbre T de hauteur h est parfait si et seulement si il a $ni = 2^h - 1$ noeuds internes.

Le niveau k d'un arbre parfait admet 2^k noeuds.

Si un arbre de hauteur h est parfait, alors $h = \ln_2(nf)$ et $h = \ln_2(ni + 1)$.

5.1.4 Parcours d'un arbre

On souhaite visiter tous les noeuds d'un arbre et appliquer une procédure (fonction qui renvoie un objet de type `unit`) à chaque noeud interne ou externe de l'arbre. Par exemple, si l'on reprend la définition des types précédente:

```
type feuille = int;;
type noeud = Plus | Fois | Max;;
type tree = F of feuille | N of tree*noeud*tree;;
```

On peut avoir envie d'afficher tous les noeuds de l'arbre, en utilisant les fonctions `traiter_feuille` et `traiter_noeud_interne` suivantes:

```
let traiter_noeud n=
match n with
Plus -> print_string (" "); print_string("plus")
|Fois -> print_string (" "); print_string("fois")
|Max -> print_string (" "); print_string("max");;

let traiter_feuille f =
print_string (" "); print_int f;;
```

a) Parcours en profondeur

La méthode la plus simple consiste à appliquer directement la fonction de traitement des feuilles si l'arbre est une feuille, et si l'arbre T à traiter est de la forme: $T = (Tg, x, Td)$, alors on appelle récursivement la fonction de parcours sur Tg (tâche 1), et Td (tâche 2) et l'on traite le noeud x (tâche 3). On a plutôt envie de faire la tâche 1 avant la tâche 2, et l'on peut choisir un ordre arbitraire pour la tâche 3, cela nous conduit aux trois fonctions suivantes:

Parcours préfixe:

```
let rec parcours_prefixe t=
  match t with
  F f-> traiter_feuille f;
  |N (tg, x, td) ->
    traiter_noeud x;
    parcours_prefixe tg; parcours_prefixe td;;
```


Parcours postfixe:

```
let rec parcours_postfixe t=
  match t with
  | F f-> traiter_feuille f;
  | N (tg, x, td) ->
    parcours_postfixe tg; parcours_postfixe td;
    traiter_noeud x;;
```

Parcours infixe

```
let rec parcours_infixe t=
  match t with
  | F f-> traiter_feuille f;
  | N (tg, x, td) ->
    parcours_infixe tg;
    traiter_noeud x;
    parcours_infixe td;;
```

Ptitexo 5. Que vont faire ces trois fonctions de parcours sur l'arbre *tex* défini, plus haut? Justifier l'appellation *parcours en profondeur*.

Pour visualiser ces parcours, on va utiliser un type d'arbre plus simple à manipuler:

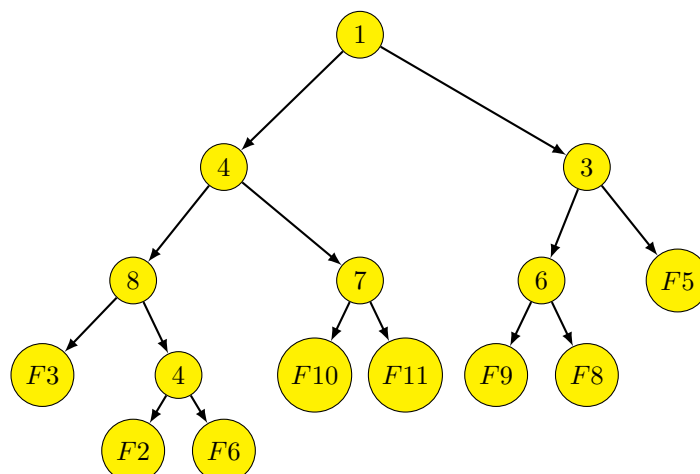
```
type arbre = Feuille of int | Noeud of arbre*int*arbre;;
```

On peut sans difficulté modifier les fonctions précédentes de parcours infixe, préfixe, postfixe de façon à ce qu'elles nous renvoient la liste des étiquettes des noeuds internes et externes dans l'ordre où sont parcourus les noeuds.

Par exemple, pour le parcours préfixe:

```
let rec liste_prefixe t=
  match t with
  | Feuille f-> [("F", f)]
  | Noeud (tg, x, td) -> ("N",x):: liste_prefixe tg@ liste_prefixe td;;
```

On va utiliser à titre d'exemple, l'arbre *texx* suivant:



Ptitexo 6. Quelle est la liste renvoyée par *prefixe texx*?

Proposition 5.21 (Unicité du parcours préfixe)

La fonction qui à un arbre binaire hétérogène associe la liste des noeuds (où l'on a distingué noeuds internes et feuilles) dans un parcours préfixe est injective.

La fonction qui à un arbre binaire hétérogène associe la liste des noeuds (où l'on a distingué noeuds internes et feuilles) dans un parcours postfixe est injective.

preuve:

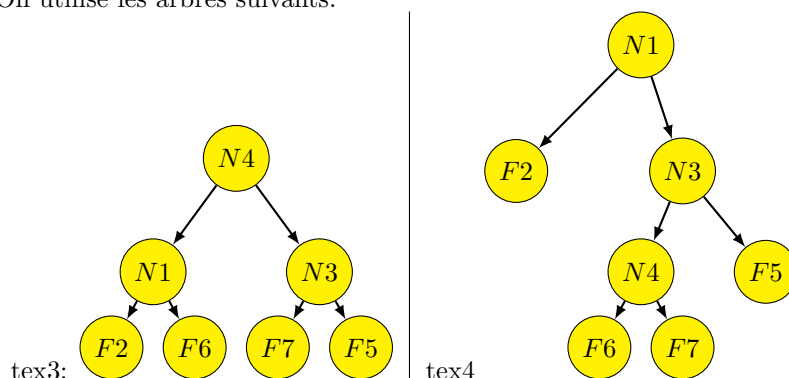
Exercice 5.1

Écrire la fonction "réciproque" de la fonction `liste_prefixe`: qui pour une liste obtenue par parcours préfixe d'un arbre renvoie l'arbre dont on est issue la liste.

On peut de manière analogue fabriquer la liste des noeuds dans le parcours infixé:

```
let rec liste_infixe t =
  match t with
  | Feuille f -> [("F", f)]
  | Noeud (tg, x, td) -> liste_infixe tg @ [("N", x)] @ liste_infixe td;;
```

On utilise les arbres suivants:



Ptitexo 7. Que renvoient `liste_infixe tex3` et `liste_infixe tex4`? Conclusion?

✓ Le parcours infixé ne décrit pas l'arbre de manière unique.

5.1.5 Parcours en largeur

Les fonctions de parcours précédentes, nous ont conduit à des parcours en profondeur: les fonctions récursives précédentes empilent les arbres à parcourir dans la pile de récursivité et on l'on visite donc d'abord les nouveaux arbres découverts.

On cherche maintenant à parcourir un arbre en largeur: c'est-à-dire que l'on veut traiter les noeuds niveau par niveau en partant de la racine.

Exercice 5.2

A partir des fonctions `traiter_noeud` et `traiter_feuille f` écrire une fonction `traiter_largeur` qui réalise le traitement en largeur, dans l'exemple précédent, la commande:

```
traiter_largeur tex
```

affichera

```
plus fois max plus max 4 plus 9 6 fois plus 3 1 1 5 7 8.
```

5.2 Arbres binaires homogènes

Dans un arbre binaire au sens de la définition précédente, tout noeud interne a exactement deux fils et toute feuille en a 0. On veut assouplir la définition de sorte qu'un noeud puisse avoir 2, 1 ou 0 fils. Dans ce but, on introduit la notion d'arbre vide: c'est la seule valeur possible pour une feuille.

Définition 5.22 (arbre binaire au sens large)

Soit $\mathcal{N}$ un ensemble. On définit un **arbre binaire au sens large** dont les noeuds sont étiquetés par $\mathcal{N}$ inductivement par:

T est un arbre si:

- T est *Vide*,
- ou, s'il existe n dans $\mathcal{N}$, G et D deux arbres tels que $T = (G, n, D)$.

✓ On parle aussi d'arbre homogène, il y a un seul type commun à tous les noeuds.

✓ Il n'y a plus de distinction entre feuilles et noeuds internes. Un arbre binaire au sens large est un arbre au sens strict avec une seule valeur possible pour les feuilles: *Vide*. C'est-à-dire que $\mathcal{F} = \{\text{Vide}\}$. L'arbre vide est aussi noté *NIL*.

Implémentation CAML: Le type de ces arbres peut-être défini par:

```
type 'a arbre = Vide | N of ('a arbre) * 'a * ('a arbre);;
```

où $'a$ désigne un type quelconque: celui commun à tous les noeuds de l'arbre.

On va, par exemple, manipuler des arbres associés à une expression arithmétique. On définit les types suivants:

```
type const = int;;
type op = Plus | Fois | Div;;
type fonc = Moins ;;
type tn = C of const | O of op | F of fonc;;
```

Le type `tn` sera le type des noeuds d'un arbre binaire au sens large représentant une expression arithmétique.

Par exemple, `let t1 = N (Nil, F Moins, N(Nil, C 3, Nil));;` permettra de représenter par `t1` l'expression -3 .

Ptitexo 8. Définir alors en CAML, les arbres T_2 représentant $e_2 = 2 * 5$ et T_3 représentant $e_3 = (-3) + (2 * 5)$. Observer le typage de ces objets.

5.2.1 Grandeurs associées à un arbre binaire homogène

✓ On laissera le lecteur formuler soigneusement les propriétés d'induction et de définition inductive des fonctions sur les arbres homogènes.

Dans toute la suite, on utilise des arbres binaires homogènes dont les noeuds sont étiquetés par un ensemble $\mathcal{N}$. On note $\mathcal{A}$ l'ensemble de ces arbres. D'un point de vue implémentation, on utilise le type `'a arbre` sus-défini.

Définition 5.23 (Noeud terminal ou externe)

On dit qu'un noeud est **terminal** ou **externe** si ses deux fils sont vides.

Un noeud non vide et non terminal est dit **noeud interne**.

Proposition 5.24 (Nombre de noeuds terminaux)

On note $nt(T)$ le nombre de noeuds terminaux de l'arbre T . On a alors:

- $nt(vide) = 0$.
- $\forall r \in \mathcal{N}, nt((vide, r, vide)) = 1$.
- $\forall r \in \mathcal{N}, \forall (G, D) \in \mathcal{A}^2 \setminus \{(vide, vide)\}, nt(G, r, D) = nt(G) + nt(D)$.

Ptitexo 9. Compléter la fonction suivante de calcul du nombre de noeuds terminaux de l'arbre T . Testez la sur les trois exemples précédents.

```
let rec nt t=
  match t with
  | Nil -> .....
  | N (Nil, _, Nil) -> .....
  | N (g, _, d) -> .....;;
```

Proposition 5.25 (Nombre de noeuds internes)

On note $ni(T)$ le nombre de noeuds internes de l'arbre T . On a alors:

- $ni(vide) = 0$.
- $\forall r \in \mathcal{N}, ni((vide, r, vide)) = 0$.
- $\forall r \in \mathcal{N}, \forall (G, D) \in \mathcal{A}^2 \setminus \{(vide, vide)\}, ni(G, r, D) = 1 + ni(G) + ni(D)$.

Définition 5.26 (Hauteur d'un arbre)

On définit de manière inductive, la hauteur $h(T)$ d'un arbre T par: $h(vide) = -1$
 $\forall (G, D) \in \mathcal{A}^2, \forall r \in \mathcal{N}, h(G, r, D) = 1 + \max(h(G), h(D))$.

✓ Comme pour les arbres binaires hétérogènes, on peut définir la profondeur d'un noeud qui est la longueur (c'est-à-dire le nombre d'arêtes) du chemin de la racine vers ce noeud.

Ptitexo 10. Écrire une fonction CAML de calcul de la hauteur d'un arbre.

Proposition 5.27 (Taille de l'arbre)

On appelle **taille de l'arbre** T le nombre de noeuds terminaux et non terminaux de cet arbre. On ne compte donc pas les noeuds vides. On a donc la définition inductive:

- $taille(vide) = 0$
- Si $r \in \mathcal{N}$ et G et D sont deux arbres, alors
 $taille(G, r, D) = 1 + taille(G) + taille(D)$.

Ptitexo 11. Écrire la fonction **taille** T de calcul de la taille d'un arbre.

Proposition 5.28 (Relation Taille-hauteur)

Soit T un arbre de taille n et de hauteur h . On suppose T non vide. On a: $h + 1 \leq n \leq 2^{h+1} - 1$.
 La hauteur d'un arbre binaire de taille n est donc comprise entre $\lfloor \ln_2(n) \rfloor$ et $n - 1$.

preuve :

On a vu qu'un arbre binaire au sens large peut être vu comme un arbre au sens strict dont les feuilles

sont les noeuds vides. Avec cette correspondance:

Les noeuds non vides de l'arbre au sens large sont les noeuds internes de l'arbre strict.

La taille n de l'arbre au sens large est donc le nombre de noeuds internes de l'arbre strict.

La hauteur h de l'arbre au sens large est la hauteur (notée h') de l'arbre strict moins 1: $h = h' - 1$ (une feuille est ici de hauteur -1 alors que dans un arbre strict elle est de hauteur 0). Dans un arbre strict, on a: $h' \leq n \leq 2^{h'} - 1$. . Cela donne la relation voulue puisque $h' = h + 1$. $\square$

Ptitexo 12. Montrer que les inégalités $h + 1 \leq n$ et $n \leq 2^{h+1} - 1$ peuvent devenir des égalités pour des arbres particuliers de hauteur arbitrairement grande.

Définition 5.29 (Arbre binaire entier, parfait)

On dit qu'un arbre binaire homogène est **entier** si chaque noeud non vide admet exactement 0 fils vides ou 2 fils vides.

On dit qu'un arbre binaire homogène est **parfait** s'il possède le nombre maximal de noeuds pour une hauteur donnée.

Proposition 5.30 (Caractérisation des arbres parfait)

Un arbre binaire homogène est complet si et seulement si il est entier et toutes ses feuilles sont de même profondeur.

Un arbre binaire parfait de hauteur h est donc de taille $2^{h+1} - 1$.

preuve:

Ptitexo 13. Écrire la fonction *recherche* x t qui renvoie *true* si x est dans t et *false* sinon.

5.2.2 Parcours d'arbres binaires homogènes

Les fonctions de parcours des arbres binaires homogènes sont très analogues à celles des arbres du chapitre précédent.

Exercice 5.3

Écrire les fonctions `parcours_infixe`, `parcours_prefixe`, `parcours_postfixe`, `parcours_largeur` pour les arbres binaires homogènes.

5.3 Introduction aux ABR

Définition 5.31 (ABR)

On appelle **arbre binaire de recherche** (ABR) un arbre binaire homogène étiqueté par un ensemble totalement ordonné $\mathcal{N}$ tel que le parcours en profondeur infixe renvoie l'énumération croissante de ses noeuds.

✓ Cela signifie qu'un arbre T est un ABR si et seulement si pour tout noeud interne étiqueté par x , les étiquettes des descendants de son fils gauche sont inférieures à x , et les étiquettes des descendants de son fils droit sont supérieures à x .

Ptitexo 14. Quels sont les arbres binaires de recherche correspondant aux valeurs 1, 2, 3 et 4?

Ptitexo 15. Écrire une fonction *verifABR* t qui renvoie *true* si et seulement si t est un ABR.

5.3.1 Lecture dans un ABR

Pour rechercher si un élément x est l'une des étiquettes d'un ABR T , il suffit de parcourir l'arbre en se limitant à chaque fois à la moitié utile de gauche ou de droite.

Exercice 5.4

Écrire une fonction de recherche d'un élément dans un ABR.
Discuter de sa complexité.

Exercice 5.5

Écrire une fonction de calcul du maximum des éléments d'un ABR. On notera que ce maximum n'est pas nécessairement une feuille.

5.3.2 Insertion, suppression dans un ABR

a) Insertion

Pour insérer un élément dans un ABR, il suffit de l'ajouter en tant que feuille à la bonne place.

Exercice 5.6

- 1) Écrire une fonction d'insertion d'un élément x dans un ABR T .
 - 2) En déduire une fonction `list2ABR li` qui transforme une liste d'éléments de type `'a` en un ABR de type `'a arbre`. Discuter de la hauteur de l'ABR, ainsi obtenu.
-

b) Suppression

Première approche:

Pour supprimer un élément x d'un ABR, on peut si c'est une feuille, la remplacer par le vide, si non, x admet un fils gauche g et un fils droit d , on peut alors supprimer x et écrire une fonction qui fusionne les deux ABR g et d en un seul ABR.

Exercice 5.7

- 1) Écrire une fonction qui fusionne deux ABR g et d en un seul ABR. Lorsqu'ils sont non vides, on supposera que $\max(g) \leq \min(d)$.
 - 2) En déduire une fonction de suppression d'un élément x dans un ABR T .
 - 3) Évaluer la complexité de cette fonction.
-

Deuxième approche: On peut supprimer un élément x , en le remplaçant par le maximum du sous-arbre de gauche de cet élément lorsque cela a un sens.

Exercice 5.8

Écrire une fonction qui supprime un élément x de l'arbre s'appuyant sur cette nouvelle démarche.

5.3.3 Rééquilibrer un ABR: rotations gauche et droite

On a déjà observé l'utilité de garder des arbres de hauteur la plus faible possible à taille donnée. Pour ce faire, il peut être utile d'introduire les fonctions de rotation gauche et droite. La rotation gauche

transforme l'arbre $(A, x, (B, y, C))$ en l'arbre $((A, x, B), y, C)$, ces deux opérations sont inverses l'une de l'autre.

Ptitexo 16. *Montrer que l'on peut rendre complet l'arbre suivant en utilisant deux rotations successives: $N(N(1, 4, N(5, 6, 7)), 8, 10)$.*

Exercice 5.9

Écrire les fonctions de rotation gauche et droite d'un ABR.
